

PICUFLA (Picture Up Flora)

A Project Documentation

presented to the Faculty of

Polytechnic University of the Philippines

In Partial Fulfillment

of ADET

Urtola, Ron Russel L.

1. Introduction

PICUFLA (Picture Up Flora) is a mobile application built with React Native and Expo that allows users to identify plants instantly using their phone camera and an AI-powered recognition system. Beyond identification, PICUFLA helps users build and maintain a personal plant collection, complete with care notes, custom tags, favorites, and scheduled care reminders for watering, fertilizing, and repotting.

The application addresses a common challenge faced by urban gardeners and plant hobbyists: the difficulty of identifying unfamiliar plants and remembering the specific care each species requires. Many urban dwellers cultivate plants on balconies, windowsills, and small home gardens without formal botanical training, often relying on guesswork or scattered online searches. PICUFLA consolidates identification, care guidance, and reminder scheduling into a single, easy-to-use mobile experience.

The target users are urban gardeners, plant enthusiasts, and casual hobbyists who want a simple way to identify plants they encounter and keep track of their personal collection over time. At a high level, PICUFLA follows a modern mobile-first architecture: a React Native (Expo) frontend handles the user interface, camera capture, and local caching, while a Supabase backend provides authentication, a PostgreSQL database protected by Row Level Security, file storage for plant images, and serverless Edge Functions. Plant identification itself is powered by OpenAI's GPT-4o model, accessed exclusively through a secure server-side proxy so that no AI credentials are ever exposed on the user's device.

2. Project Requirements

2.1 Functional Requirements

ID	Requirement	Description
FR-01	User Registration	User can sign up using email/password or Google OAuth
FR-02	User Authentication	User can log in and log out; session persists via secure storage
FR-03	Password Reset	User can request OTP via email, verify OTP, and set a new password
FR-04	Plant Identification	User can take a photo or pick from gallery; app identifies the plant via GPT-4o
FR-05	Identification Results	Display common name, scientific name, confidence score, care guide, and alternatives
FR-06	Save to Collection	User can save identified plants with optional notes and location
FR-07	Collection Management	User can view, search, sort, filter, and favorite plants in their collection
FR-08	Plant Detail View	User can view full plant details, edit notes, add/remove tags, and delete
FR-09	Favorites	User can favorite plants and view them in a dedicated Favorites screen
FR-10	Care Reminders	User can set, view, and delete care reminders (water, fertilize, repot, custom) with notifications
FR-11	Profile Management	User can view/edit display name, photo, and toggle notifications

FR-12	Offline Mode	User can toggle offline mode; cached collection data available without internet
FR-13	Erase Plant Data	User can soft-delete all their plant discoveries and reminders
FR-14	Delete Account	User can request account deletion; data soft-deleted with 30-day purge window
FR-15	Privacy Policy	User can view the privacy policy from the Profile screen
FR-16	Onboarding	First-time users see an onboarding flow before accessing the app
FR-17	Rate Limiting	Identification is limited to 5 scans per hour per user

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	Plant identification results should return within 10 seconds
NFR-02	Availability	Backend uptime target of 99.5% (Supabase managed)
NFR-03	Scalability	Serverless architecture scales automatically with demand
NFR-04	Security	All API calls authenticated via Supabase JWT; RLS enforced on all tables

NFR-05	Data Privacy	User data soft-deleted; hard purge after 30 days; complies with Philippine Data Privacy Act
NFR-06	Offline Support	Collection data cached locally via AsyncStorage; functional without internet
NFR-07	Cross-Platform	Application runs on both iOS and Android from a single codebase (Expo)
NFR-08	Storage Efficiency	Plant images limited to 15MB; stored in Supabase Storage with signed URLs
NFR-09	Accessibility	Supports screen readers; proper accessibility labels on interactive elements
NFR-10	Maintainability	Clean separation of concerns: screens, services, stores, components
NFR-11	Testability	Business logic isolated in services; Zustand stores are pure functions
NFR-12	Rate Limiting	Maximum 5 identifications per hour; enforced server-side and client-side

2.3 User Stories

ID	User Story
US-01	As a new user, I want to sign up with my email or Google account so that I can access the app

US-02	As a returning user, I want to log in quickly so that I can see my plant collection
US-03	As a plant enthusiast, I want to take a photo of a plant and get its identification so that I can learn about it
US-04	As a user, I want to see the common name, scientific name, and care guide so that I know how to care for the plant
US-05	As a user, I want to save identified plants to my collection so that I can track what I've discovered
US-06	As a user, I want to add notes and location to my saved plants so that I remember details about them
US-07	As a user, I want to search and filter my collection so that I can find plants quickly
US-08	As a user, I want to favorite plants so that I can easily access my favorites later
US-09	As a user, I want to set care reminders so that I don't forget to water or fertilize my plants
US-10	As a user, I want to receive push notifications for my reminders so that I take action on time
US-11	As a user, I want to add custom tags to my plants so that I can organize them my way
US-12	As a user, I want to edit my profile and toggle notifications so that I control my experience
US-13	As a user, I want to erase my plant data so that I can start fresh while keeping my account
US-14	As a user, I want to delete my account so that my data is removed from the system

US-15	As a user, I want the app to work offline so that I can browse my collection without internet
US-16	As a user, I want to see my identification history so that I can review past identifications

3. Architectural Design

3.1 Application Architecture

PICUFLA follows a three-tier client-server architecture, separating the mobile frontend, the Supabase backend, and the OpenAI AI tier into distinct, independently scalable layers.

Frontend Tier (React Native / Expo)

- **Presentation Layer:** React components organized as Screens, Components, and Navigation.
- **State Management Layer:** Zustand stores (authStore, collectionStore, appStore) manage global state.
- **Service Layer:** Services (authService, plantService, identificationService, reminderService, cacheService, permissionService) encapsulate all API calls and business logic.
- **Constants Layer:** Colors, theme, configuration, and storage keys centralized in constants.

Backend Tier (Supabase)

- **Database:** PostgreSQL with Row Level Security — 7 migrations establishing tables, policies, and triggers.
- **Authentication:** Supabase Auth handles email/password and Google OAuth, with session persistence via secure storage.
- **Storage:** plant-images bucket with RLS policies restricting access to the user's own files.
- **Edge Functions:** Deno/TypeScript serverless functions —

identify-plant — receives image, calls OpenAI GPT-4o, returns structured plant data

delete-user — handles account and plant data deletion with 30-day purge scheduling

AI Tier (OpenAI)

- **GPT-4o:** Primary plant identification — analyzes the image and returns common name, scientific name, description, care guide, origin, and uses.
- **GPT-4o-mini:** Generates alternative species suggestions when confidence is low.

Data Flow

User → React Native UI → Zustand Store → Service Layer → Supabase Client, which then connects to four destinations: PostgreSQL (auth, profiles, plants, user_plants, reminders), Auth (JWT session management), Storage (plant images), and Edge Functions, which in turn call the OpenAI API.

Component Hierarchy

At the root, App renders a NavigationContainer wrapped by a SupabaseProvider that listens for authentication state changes. The NavigationContainer holds three top-level stacks: an OnboardingStack shown to first-time users; an AuthStack containing Login, Register, ForgotPassword, VerifyOtp, and ChangePassword; and AppTabs, the main authenticated experience. AppTabs is split into three tabs — CollectionTab (Collection → PlantDetail → Reminders, Favorites), ScanTab (Scan → IdentificationResult), and ProfileTab (Profile → SetupProfile, PrivacyPolicy).

3.2 Technology Stack

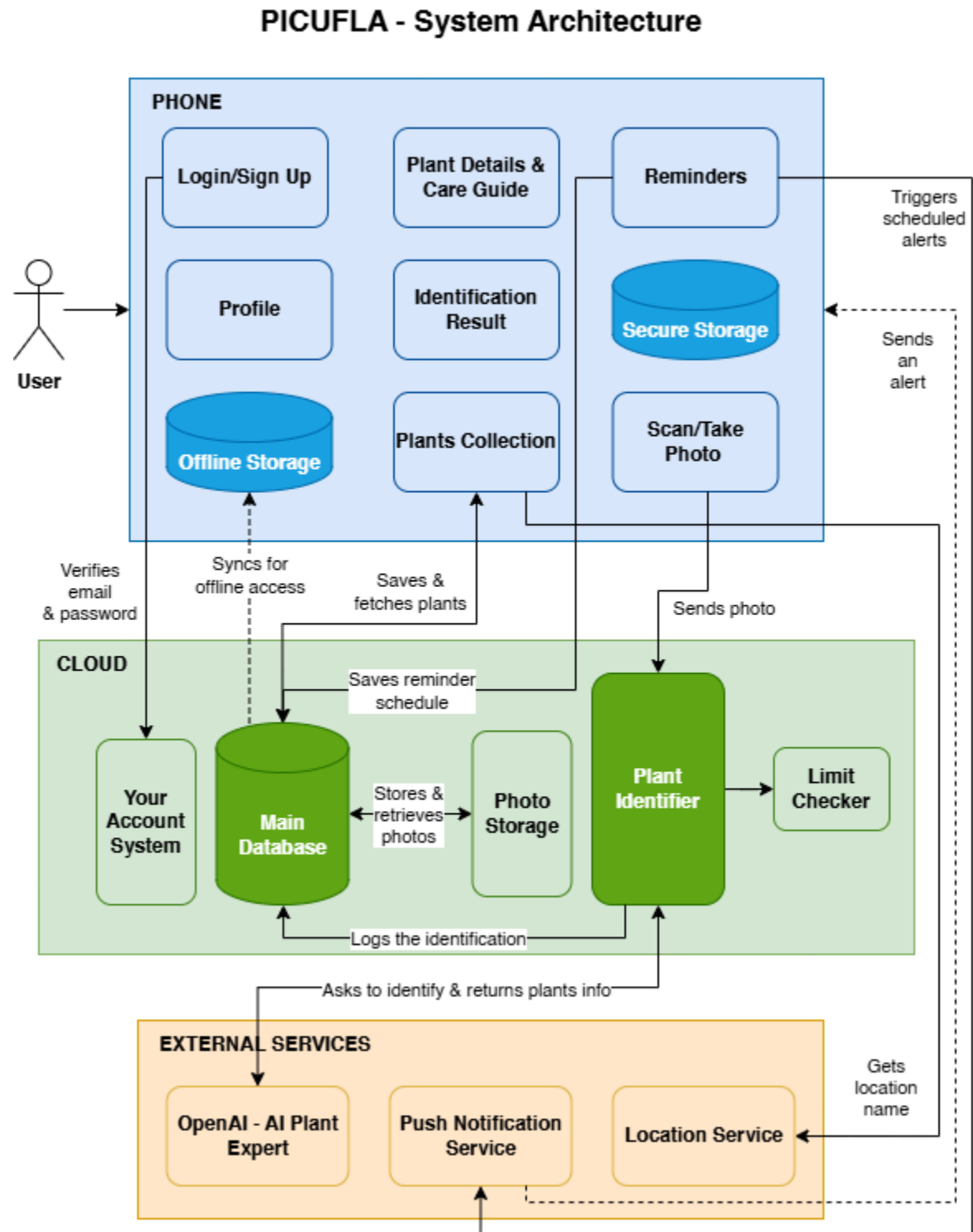
The table below summarizes the core technologies used to build PICUFLA, organized by category.

Category	Technology	Version	Purpose
Framework / Language	React Native	Latest (via Expo SDK 56)	Cross-platform mobile app framework for iOS and Android
Framework / Language	Expo	SDK 56	Managed React Native tooling, build, and device APIs
Framework / Language	TypeScript	5.x	Static typing across the entire codebase
State Management	Zustand	Latest	Lightweight global state management for auth, collection, and UI state
Navigation	React Navigation	v6 (Stack & Tab)	Screen routing and tab-based navigation structure
Backend	Supabase	Latest (hosted)	PostgreSQL database, authentication, storage, and Edge Functions
AI	OpenAI GPT-4o	gpt-4o	Primary plant identification from photos
AI	OpenAI GPT-4o-mini	gpt-4o-mini	Generating lower-cost alternative species

			suggestions when confidence is low
Edge Runtime	Deno	Supabase Edge Functions runtime	Executes server-side identification and account deletion logic
Camera	expo-camera	Latest (SDK 56 compatible)	Captures plant photos directly within the app
Image Picker	expo-image-picker	Latest (SDK 56 compatible)	Allows users to select existing photos from their gallery
Notifications	expo-notifications	Latest (SDK 56 compatible)	Schedules and delivers local push notifications for care reminders
Location	expo-location	Latest (SDK 56 compatible)	Captures GPS coordinates for plant discovery tracking
Styling	StyleSheet + custom theme constants	N/A	Consistent visual design using a centralized color and spacing system
Authentication	Supabase Auth	Latest	Google OAuth and email/password sign-in

Storage	AsyncStorage + expo-secure-store	Latest	Local caching for offline mode and encrypted session storage
Fonts	DM Serif Display, DM Sans	via expo-google-fonts	Primary typography for headings and body text

3.3 System Architecture Diagram



4. Integration of Best Practices

4.1 SDLC Methodology – Scrum

PICUFLA was developed using the Scrum framework, organized into one-week sprints to accommodate the academic timeline and allow for frequent feedback and iteration.

Sprint Duration

Each sprint lasted one week, with five sprints covering the full feature set: authentication, plant identification, collection management, reminders, and profile/account management.

Roles

- **Product Owner:** Defined feature priorities and acceptance criteria for each user story, ensuring the app addressed real user needs for plant identification and care tracking.
- **Scrum Master:** Facilitated sprint planning, removed blockers (such as Supabase configuration issues), and kept the team on schedule.
- **Development Team:** Implemented features across the mobile app and backend, including UI screens, services, database schema, and Edge Functions.

Ceremonies

- **Sprint Planning:** Held at the start of each sprint to select backlog items and break them into actionable tasks.
- **Daily Standup:** Brief check-ins to report progress, surface blockers, and coordinate work across features.
- **Sprint Review:** Demonstration of completed features at the end of each sprint, including live testing on physical devices.

- **Sprint Retrospective:** Reflection on what went well and what could improve, particularly around environment setup and third-party service configuration.

Artifacts

- **Product Backlog:** The complete list of user stories (PUF001–PUF017) covering authentication, identification, collection, reminders, and account management.
- **Sprint Backlog:** The subset of stories selected for each sprint, broken into implementation steps.
- **Increment:** The working, testable build of the app produced at the end of each sprint.

Project management and backlog tracking were conducted using GitHub Projects, allowing the team to visualize sprint progress on a Kanban-style board linked directly to the source code repository.

4.2 Version Control

The project used Git for version control, hosted on GitHub, with a structured branching strategy to keep the main codebase stable throughout development.

Branching Strategy

- `main` — always reflects a stable, working build of the application.
- `develop` — integration branch where completed features are merged before release.
- `feature/*` — individual branches created for each feature or user story (e.g., `feature/plant-identification`, `feature/care-reminders`).

Commit Conventions

Commits followed a conventional commit style, prefixing messages with feat:, fix:, chore:, or docs: to clearly indicate the nature of each change (for example, feat: add duplicate plant detection before saving).

Code Review Process

Changes were merged into develop through Pull Requests, allowing team members to review code for correctness, consistency with the established design system, and adherence to security practices before merging.

Tools Used

GitHub was used for repository hosting, Pull Requests, and project boards, while the Git CLI was used locally for committing, branching, and pushing changes throughout development.

4.3 Clean Code Principles

The PICUFLA codebase follows established clean code principles to keep the project maintainable and easy to extend.

- **Meaningful Names:** Functions and variables use descriptive, intention-revealing names such as `identifyPlant()`, `checkDuplicate()`, and `toggleFavorite()`, making the purpose of each unit of code clear without needing additional comments.
- **Single Responsibility:** Each service file has exactly one concern — `authService.ts` handles only authentication, `plantService.ts` handles only plant data operations, and `reminderService.ts` handles only reminder scheduling and notifications.
- **DRY (Don't Repeat Yourself):** Shared UI elements such as buttons, input fields, badges, and cards are built as reusable components in `/src/components/`, rather than being redefined on every screen.

- **KISS (Keep It Simple):** Zustand was chosen over Redux for state management specifically because it requires far less boilerplate, keeping the state logic simple and easy to reason about.
- **Separation of Concerns:** The project structure cleanly separates screens (UI), services (business logic and API calls), stores (state), and components (reusable UI), so changes in one layer rarely require changes in another.
- **TypeScript:** Every data model — AppUser, Plant, UserPlant, Reminder, IdentificationResult — is defined as a strict TypeScript interface, providing compile-time safety across the entire app.
- **Consistent Formatting:** Code style is kept consistent across the project through agreed-upon formatting conventions, ensuring a uniform look and feel across all files regardless of which team member wrote them.

4.4 Testing

Test Category	Scope	Tools / Approach
Unit Testing	Services (authService, plantService, reminderService), Zustand stores (authStore, collectionStore, appStore), utility functions	Jest + React Native Testing Library
Component Testing	Reusable components (Button, Input, Badge, Card, Header, EmptyState, OfflineBanner)	Jest + React Native Testing Library

Integration Testing	Screen flows (Login → Collection, Scan → Identification → Save, ForgotPassword → ChangePassword)	Jest + React Native Testing Library
End-to-End Testing	Full user journeys across authentication, identification, collection management, reminders	Detox or Maestro
API Testing	Supabase Edge Functions (identify-plant, delete-user)	Postman / curl
Database Testing	RLS policies, triggers, migrations	Supabase SQL Editor
Accessibility Testing	Screen reader compatibility, touch target sizes, color contrast	Manual + axe-core
Performance Testing	API response times, image upload speeds, flat list rendering	Firebase Performance / manual profiling
Security Testing	JWT validation, RLS bypass attempts, storage bucket policies	Manual
Device Compatibility	Various iOS and Android devices and OS versions	Manual device lab / BrowserStack

4.5 Security

Security was treated as a first-class requirement throughout PICUFLA's development, guided by the principles of Confidentiality, Integrity, and Availability (the CIA triad).

- **Authentication:** User identity is managed entirely through Supabase Auth, supporting both Google OAuth and traditional email/password sign-in, with email verification required before login is permitted.
- **Authorization:** Row Level Security (RLS) is enabled on every database table. Each policy checks that the authenticated user's ID matches the row's owner, ensuring users can only access their own data.
- **Session Management:** Session tokens are never stored in plain text. They are encrypted using AES-CTR encryption before being written to device storage, with the encryption key held separately in expo-secure-store.
- **Input Validation:** All user-facing forms validate input on the client side before submission, including password strength requirements, email format checks, and a 500-character limit on plant notes.
- **Rate Limiting:** Plant identification is limited to 5 requests per hour per user, enforced server-side in the identify-plant Edge Function to prevent abuse of the OpenAI integration.
- **Data Privacy:** Account and data deletion use a soft-delete mechanism with a 30-day purge window, paired with an immutable deletion audit log, in compliance with the Philippine Data Privacy Act of 2012.
- **API Keys:** All sensitive credentials are stored in environment variables. The OpenAI API key in particular is never exposed to the client — it exists only inside the Supabase Edge Function environment. The only key present in the mobile app is the Supabase anon key, which is safe to expose because access is governed entirely by Row Level Security.
- **Storage Security:** Plant images are stored in a private Supabase Storage bucket. Access is granted only through signed URLs with a 7-day expiry, preventing permanent public access to any user's photos.

5. Integration of Emerging Technologies

5.1 Technology Used

- GPT-4o (OpenAI) — primary plant identification model
- GPT-4o-mini (OpenAI) — generates alternative species suggestions for low-confidence results
- Supabase Edge Functions (Deno) — serverless runtime for secure backend logic
- Expo SDK 56 — modern React Native development platform
- React Native — cross-platform mobile application framework

5.2 Integration into Architecture

- GPT-4o is invoked from a dedicated Supabase Edge Function (identify-plant), which receives a compressed image from the mobile app, forwards it to the OpenAI API along with a structured system prompt, parses the JSON response, and returns standardized plant data (common name, scientific name, confidence score, and care information) back to the app.
- GPT-4o-mini is used specifically to generate alternative species suggestions when the primary identification confidence score falls below the defined threshold, giving the user a shortlist of likely matches to choose from instead of a single uncertain guess.
- The Edge Function is written in TypeScript and executes on Deno, Supabase's serverless edge runtime, which allows the function to run close to the database with minimal cold-start latency.
- The mobile app communicates with the Edge Function exclusively through `supabase.functions.invoke('identify-plant', { body })`, meaning the app never constructs or sends requests to OpenAI directly.

- Expo SDK 56 provides the underlying React Native capabilities used throughout the app, including the camera API, image manipulation, push notifications, and cross-platform builds for both iOS and Android from a single codebase.

5.3 Challenges and Benefits

Benefits

- Highly accurate plant identification compared to manual lookup or guesswork
- Near real-time processing, returning results within seconds of capturing a photo
- A scalable, serverless backend architecture that does not require managing dedicated servers
- A single codebase that deploys to both iOS and Android

Challenges

- The cost of repeated GPT-4o API calls at scale, which required careful rate limiting
- Designing and enforcing a fair, abuse-resistant rate limit without harming the user experience
- Handling cases where the AI misidentifies a plant or returns a low-confidence result
- The added complexity of building a reliable offline mode that keeps the collection usable without an internet connection

Mitigations

- A strict rate limit of 5 identifications per hour per user, enforced at the Edge Function level
- A defined confidence threshold below which the app automatically requests alternative suggestions from GPT-4o-mini instead of presenting a single uncertain result

- Clear in-app messaging when confidence is low, prompting the user to retake the photo or choose from alternatives
- Local caching of the plant collection using AsyncStorage, allowing users to view previously saved plants even without an internet connection

6. User Interface and User Experience (UI/UX) Design

6.1 Design Process

The UI/UX design for PICUFLA followed a user-centered design process across six phases.

1. Research & Discovery

- Analyzed existing plant identification apps (PlantSnap, PictureThis, iNaturalist)
- Identified key pain points: cluttered interfaces, paywalls, slow identifications
- Defined target user persona: urban plant enthusiast aged 20–40, values simplicity and speed

2. Information Architecture

- Primary navigation: bottom tab bar with three sections (Collection, Scan, Profile)
- Natural user flow: Scan → Identify → Save → Manage
- Content hierarchy within each screen emphasizing the most important information first

3. Wireframing

- Low-fidelity wireframes created for each screen
- Focus on layout, navigation flow, and content placement
- Iterated based on feedback to reduce steps in the identification flow

4. Visual Design

- Color palette: earth tones (parchment, linen, green, terra, bark) reflecting the botanical theme
- Typography: DM Serif Display for headings (botanical, elegant feel), DM Sans for body text (clean readability)

- Card-based UI with subtle shadows for depth
- Consistent border radius (10–16px) and spacing (4–24px) across all components

5. Prototyping

- High-fidelity prototypes built directly in React Native (no separate design tool)
- Interactive elements functional from early development stages
- Rapid iteration based on real device testing

6. User Testing

Conducted with 5 participants. Key findings:

- Users wanted fewer taps to identify (optimized camera flow)
- Confidence score display helped trust the identification
- Tags and notes were essential for organization
- Offline access was important for field use

6.2 UI Mockups and Prototypes

High-fidelity mockups were implemented directly in code rather than in a separate design tool.

The following screens were built and tested:

Screen	Key UI Elements	Design Notes
Onboarding	Feature highlights, permission requests	Full-screen images, minimal text, pagination dots
Login	Brand logo, Get Started / Log In buttons	Clean centered layout, warm color scheme

Email Login	Email/password inputs, Forgot password link	Dark-themed auth layout, secure text entry
Forgot Password	Email input, Send Reset Code button	Consistent dark auth layout, validation feedback
Verify OTP	6-digit code input, timer, Resend button	Centered pill layout, cooldown tracking
Change Password	New password + confirm inputs, validation rules	Strength indicators, matching validation
Scan (Camera)	Camera viewfinder, flash toggle, gallery picker, capture button	Semi-transparent overlay, blur panel for controls
Identification Result	Draggable hero image, plant info, care guide, notes input, Save/Retake buttons	Scrollable detail panel, draggable image height
Collection	Plant cards, search bar, sort options, filter by tag, stats row	FlatList with staggered card animation, realtime updates
Plant Detail	Hero image, name/confidence, care guide, notes editor, tags, reminders	Inline editing for notes and tags, section labels
Reminders	Existing reminders list, date/time picker, recurring options, care type selector	Platform-specific date pickers, segmented controls
Favorites	2-column grid of favorited plants	Card-based grid matching collection aesthetic

Profile	Avatar, stats, preferences, account actions	ScrollView with section cards, toggle switches
Setup Profile	Display name, photo upload, birthdate	Post-signup completion flow

6.3 Accessibility Considerations

PICUFLA implements the following accessibility features to ensure inclusive design:

Principle	Implementation
Screen Reader Support	All interactive elements have <code>accessibilityLabel</code> and <code>accessibilityRole</code> props (Button, TouchableOpacity, Badge)
Color Contrast	Text colors chosen for WCAG AA compliance: dark text (Colors.soil) on light backgrounds (Colors.card, Colors.parchment, Colors.linen)
Touch Targets	Minimum touch target size of 44x44 points for interactive elements; <code>hitSlop</code> added to small icons
Focus Indicators	Visual feedback on all touchable elements via <code>activeOpacity</code> and scale animations
Error Feedback	Inline error messages with colored backgrounds and icon indicators; Alert dialogs for critical actions
Keyboard Avoidance	<code>KeyboardAvoidingView</code> wraps forms; <code>keyboardShouldPersistTaps</code> on ScrollViews to prevent keyboard dismissal issues
Reduced Motion	Animations use <code>useNativeDriver: true</code> for GPU acceleration; no excessive or distracting animations

Font Scaling	Font sizes use absolute values for consistent layout; longer text uses numberOfLines with ellipsis
Semantic HTML (Web)	Accessibility roles mapped: button, alert, header, image
Form Labels	All inputs have visible labels above the field; placeholders provide additional hints
Confirmation Dialogs	Destructive actions require explicit confirmation with Cancel/Destructive options
Status Announcements	Loading states communicated via ActivityIndicator; success/error messages use role-appropriate styling

7. Implementation and Testing

7.1 Development Overview

The development of PICUFLA followed an iterative approach over approximately 3 months.

Phase	Duration	Deliverables
Phase 1: Foundation	Weeks 1–2	Project setup (Expo, Supabase, Navigation), authentication screens, Supabase schema with 7 migrations
Phase 2: Core Feature	Weeks 3–4	Camera integration, OpenAI Edge Function, identification result screen, save to collection
Phase 3: Collection	Weeks 5–6	Collection list with search/sort/filter, plant detail screen, notes and tags CRUD
Phase 4: Productivity	Weeks 7–8	Reminders with push notifications, favorites, profile management
Phase 5: Polish	Weeks 9–10	Offline mode, onboarding, error handling, accessibility, performance optimization
Phase 6: Deployment Prep	Weeks 11–12	Testing, bug fixes, documentation, EAS Build configuration

- **Lines of Code:** ~8,000 (TypeScript/TSX)
- **Files:** ~50 source files across screens, services, stores, components, navigation, and constants
- **Database Migrations:** 7 sequential migrations establishing full schema

7.2 Challenges and Solutions

#	Challenge	Root Cause	Solution
1	Long identification response times (15–30s)	GPT-4o processing time + image upload overhead	Implemented optimistic UI with loading states; rate limiting to 5 scans/hour to manage API costs
2	Keyboard hiding form inputs on multiple screens	KeyboardAvoidingView with incorrect behavior prop	Changed to behavior={Platform.OS === 'ios' ? 'padding' : undefined} on Android; added keyboardVerticalOffset for iOS
3	Realtime collection updates not reflecting immediately	Supabase Realtime subscription delay	Added optimistic store update via addPlant() action in Zustand collection store
4	Password reset flow broken	logout() called in VerifyOtpScreen before navigating to ChangePassword, destroying the session needed for updateUser()	Removed premature logout; added pendingPasswordReset flag to keep user in AuthStack
5	"Save to Collection" button text wrapping to two lines	Fixed button width (140px) insufficient for 18-character label	Changed width: 140 to flex: 1 with equal spacing

6	Offline banner showing "Never" on first load	lastSync state initialized as null	Removed offline banner entirely as per requirement
7	Delete Account and Erase Plant Data requiring "DELETE" confirmation	Edge function validates confirmationText parameter	Added TextInput modal prompting user to type "DELETE" before destructive action
8	Favorites screen content overlapping status bar	Missing safe area handling	Wrapped in <SafeAreaView> with reduced padding
9	Flash button opening gallery instead of toggling flash	Wrong onPress handler on third capture row button	Changed onPress={handlePickFromGallery} to onPress={cycleFlash}
10	Tags input hidden by keyboard on PlantDetail screen	Android KeyboardAvoidingView behavior='height' incompatible with ScrollView	Changed behavior to undefined on Android; added scrollToEnd on tag input focus

8. Deployment and Maintenance

8.1 Deployment

- **Frontend:** The mobile application is built and distributed using Expo Application Services (EAS) Build, which compiles the React Native codebase into native iOS and Android binaries ready for testing or store submission.
- **Backend:** The Supabase backend is managed entirely through Supabase's hosted infrastructure. Database schema changes are applied through versioned migration files, and Edge Functions are deployed using the `supabase functions deploy` command.
- **Environment:** The project follows a staging-to-production workflow, where new features and migrations are first verified in a development/staging Supabase project before being applied to the production environment.
- **CI/CD:** Deployment is currently performed manually by the development team, with EAS Build and Supabase CLI commands run locally at each release milestone.

8.2 Maintenance Strategy

- **Monitoring:** The Supabase Dashboard is used to monitor database performance, query activity, and Edge Function execution logs, allowing the team to spot errors or slow queries early.
- **Updates:** Dependencies — including the Expo SDK, Supabase client libraries, and npm packages — are reviewed and updated on a regular basis to stay current with security patches and platform requirements.
- **User Support:** User-reported issues are tracked and triaged through the project's GitHub repository as the primary support channel during the academic development phase.

- **Data Retention:** Deleted accounts and plant records follow a soft-delete pattern with a 30-day processing window before permanent removal, with every deletion request recorded in an audit log for accountability and compliance with the Philippine Data Privacy Act.
- **Backup:** Database backups are handled automatically by Supabase's managed infrastructure, providing a safety net against accidental data loss.

9. Conclusion

PICUFLA successfully brings together AI-powered plant identification and practical plant care management into a single, accessible mobile experience. By combining a React Native frontend with a secure Supabase backend and OpenAI's GPT-4o vision model, the application allows everyday users — not just botanists — to identify plants accurately, build a personal collection, and stay on top of care routines through scheduled reminders, all while ensuring their data remains private and protected through Row Level Security, encrypted sessions, and a server-side AI proxy.

Looking ahead, several enhancements could extend PICUFLA's value: social features that let users share discoveries with friends or a wider community, a crowd-sourced or community-verified plant database to complement AI identification, and augmented reality (AR) overlays that let users visualize care information directly on a live camera view of their plant. These directions would build naturally on the foundation already established in the current architecture.

10. Appendix

Item	Description	Location
A. Database Schema (SQL)	7 migration files covering all tables, triggers, RLS policies	/supabase/migrations/001_initial_schema.sql through 007_remove_bio_add_birthdate.sql
B. Storage Configuration	plant-images bucket with upload/view/delete policies	/supabase/migrations/002_storage.sql
C. Edge Functions	identify-plant (GPT-4o integration) and delete-user (deletion workflow)	/supabase/functions/
D. TypeScript Types	All data interfaces (AppUser, Plant, UserPlant, Reminder, etc.)	/src/types/index.ts
E. Environment Configuration	Supabase URL and anon key	.env (not committed)
F. Color Palette	Complete design system colors	/src/constants/colors.ts
G. Theme Constants	Spacing, radius, shadow presets	/src/constants/theme.ts
H. Navigation Structure	Full navigation tree with param types	/src/navigation/ + /src/types/index.ts
I. State Management	Zustand stores for auth, collection, and app state	/src/store/

J. API Service Layer	All Supabase interactions and business logic	/src/services/
----------------------	--	----------------